

Proyectos reales con IA

De la idea a un repositorio que se vea en tu GitHub y en tu CV, trabajando con OpenCode y Claude Code

- **Parte 1.** Cómo enfocar un proyecto personal para GitHub y para el CV
- **Parte 2.** Tres ideas gratuitas, sencillas y con recorrido
- **Parte 3.** Cómo turnar OpenCode y Claude Code sin pisarse
- **Parte 4.** AGENTS.md y MEMORY.md en todos tus proyectos, paso a paso
- **Parte 5.** Control de versiones y CI/CD desde el primer día
- **Parte 6.** Checklist final y siguiente paso

Sobre las fuentes. No he visto el vídeo del curso; solo he usado su descripción (los errores de la IA suelen empezar en una petición ambigua o un contexto incompleto; conviene convertir la idea en un plan con restricciones y evaluar la propuesta antes de generar código). Los criterios de elección de modelos salen de la *Guía de modelos de IA 2026* del curso. Todo lo demás es propuesta mía: compruébalo antes de darlo por bueno, sobre todo las versiones de herramientas, que cambian rápido.

Parte 1. Enfocar un proyecto para GitHub y para el CV

Un reclutador dedica menos de un minuto a un repositorio. Lo que convence no es el tamaño, es que se vea que **resuelve algo concreto, funciona y está cuidado**. La IA acelera el código, pero no puede poner tu criterio: eso es lo que se evalúa.

Paso 1. Elige un problema que puedas explicar en una frase

Fórmula: «Ayuda a [quién] a [hacer qué] sin [qué dolor]». Si no cabe en una frase, el alcance es demasiado grande. Prefiere un proyecto pequeño terminado a uno grande a medias.

Paso 2. Define qué significa «terminado» antes de empezar

Es el consejo más útil de la guía de modelos: un criterio verificable, por ejemplo «funciona la instalación en limpio, pasan los tests y hay una demo pública». Sin criterio, la IA no sabe cuándo parar y tú tampoco.

Paso 3. Convierte la idea en plan antes de generar una línea

Pide a la IA que planifique, discute el plan y solo entonces implementa. Este es el **prompt de arranque** que te propongo (es mío, no el del vídeo). Pégalo tal cual en Claude Code o en OpenCode cambiando lo que va entre corchetes:

```
Actúa como líder técnico de este proyecto. NO escribas código todavía.

IDEA: [una frase con el problema y para quién]
RESTRICCIONES:
- Coste 0 EUR: solo servicios con plan gratuito (GitHub, Actions, Pages...).
- Stack: [p. ej. TypeScript + Node 22] salvo que justifiques otro.
- Debe poder ejecutarse en Windows y Linux.
- Licencia MIT. Sin secretos en el repositorio.
- Alcance MVP: como máximo 2 semanas a ratos.

RESPONDE CON:
1. Cómo entiendes el problema y tus supuestos.
2. Hasta 5 preguntas que necesites que te resuelva.
3. Qué entra y qué NO entra en el MVP.
4. Criterio de "terminado" verificable.
5. Dos arquitecturas posibles con pros y contras, y cuál recomiendas.
6. Plan en hitos pequeños; cada hito con su prueba.
7. Riesgos y cómo los reducimos.

Espera mi aprobación antes de generar nada.
```

Paso 4. Evalúa la propuesta como lo haría un revisor

- ¿Cada hito se puede probar de forma automática?
- ¿Hay algo que no entiendes? Pídele que te lo explique hasta que puedas contarlo tú. En la guía de modelos es la pregunta clave: «¿puedo explicarlo?».
- ¿Ha añadido dependencias o servicios de pago que no pediste? Quítalos ahora, no después.

Paso 5. Construye en hitos pequeños con tests y commits pequeños

Un hito, una rama, un commit o pocos, tests en verde, integración. La historia de commits limpia es parte del portfolio: enseña cómo trabajas.

Paso 6. Cuida el repositorio como si fuera un producto

Elemento	Qué poner	Por qué importa
README	Una frase de qué es, captura o GIF, instalación en 3 comandos, ejemplo de uso, enlace a la demo.	Es lo único que leen la mayoría.
Descripción y topics	Frase corta + 5-8 topics (typescript, cli, github-actions...).	Aparece en buscadores y en tu perfil.
Licencia	MIT (o la que elijas) en LICENSE.	Sin licencia nadie puede usarlo.
CI	Badge de estado de Actions en el README.	Muestra que se prueba solo.
Releases	Tags v0.1.0, v0.2.0 con notas de versión.	Indica que el proyecto vive.
Demo	GitHub Pages o el paquete publicado (npm).	Se puede probar sin instalar nada.
docs/decisiones	Una nota corta por decisión importante (qué elegiste y por qué).	Demuestra criterio, que es lo que buscan.
AGENTS.md / MEMORY.md	Ver Parte 4.	Enseña que sabes trabajar con IA de forma ordenada.

Paso 7. Perfil de GitHub y CV

- **README de perfil:** crea un repositorio público con el mismo nombre que tu usuario (BertMarti/BertMarti) y su README.md aparecerá en tu perfil. Pon quién eres, tu stack, 3 proyectos destacados con enlace y cómo contactarte.
- **Fija (pin) 4-6 repositorios** en tu perfil: solo los que estén terminados y con buen README.
- **En el CV**, una línea por proyecto con formato *acción + tecnología + resultado*. Ejemplo: «Desarrollé una CLI en TypeScript publicada en npm con CI multiplataforma (Windows/Linux) y despliegue automático de documentación; X descargas en el primer mes».
- **Sé honesto con la IA:** di que trabajaste asistido por IA y qué decisiones tomaste tú. Es lo normal y a favor tuyo si puedes explicar el código en una entrevista.

Cuidado con lo que sube a un repositorio público. El de este curso es público. Nunca subas claves de API (Google, OpenAI...), archivos .env ni código de tu empresa. Si una clave llega a subirse, cámbiala: borrar el commit no basta.

Parte 2. Tres ideas gratuitas, sencillas y con recorrido

Criterios: **coste cero ahora** (GitHub, Actions y Pages son gratuitos en repositorios públicos), alcance de un MVP en pocas semanas, y una vía realista de generar ingresos más adelante. «Realista» no es «seguro»: son hipótesis que se validan con usuarios reales.

	1. ai-project-kit	2. Selector de modelos y costes	3. Portfolio desde GitHub
Qué es	CLI y plantilla que crea proyectos listos para trabajar con IA (AGENTS.md, MEMORY.md, CI...).	Web estática que compara modelos de IA y calcula cuánto cuesta cada tarea.	Acción de GitHub que genera tu portfolio y CV a partir de tus repos públicos.
Stack gratis	TypeScript, npm, Actions, Pages	Astro o Vite, JSON versionado, Actions, Pages	Node, API de GitHub, Actions, Pages, PDF local
Esfuerzo MVP	Bajo	Bajo-medio	Medio
Para tu CV	Paquete publicado en npm, tests, CI multiplataforma	Web pública, datos con validación, automatización mensual	Acción reutilizable, uso de API, generación de PDF
Dinero a futuro	GitHub Sponsors, packs de plantillas de pago, consultoría	Anuncios/afiliación, newsletter, API de datos de pago	Temas premium, dominio propio, versión hospedada
Riesgo principal	Ya existen generadores; hay que aportar una opinión clara.	Los precios cambian: exige mantenimiento y fuentes citadas.	Hay mucho ya hecho; diferénciate.

Idea 1. ai-project-kit (la más útil para ti desde el primer día)

Tú mismo vas a necesitar esto en cada proyecto, así que construirlo tiene retorno inmediato. Un comando `ai-project-kit init` genera la carpeta con AGENTS.md, CLAUDE.md, MEMORY.md, workflows de CI, plantillas de issues y PR, .gitignore y un README base, según el stack elegido (Node o Python).

- **MVP:** comando `init`, dos plantillas (Node y Python), tests de que los archivos se generan bien.
- **Publicación:** paquete en npm (gratis) y repositorio marcado como «Template repository».
- **Futuro:** más stacks, plantillas de pago, tutorial en vídeo o artículo, patrocinio.

Idea 2. Selector de modelos y calculadora de costes

La guía del curso enseña a elegir modelo según tarea y a calcular el coste con la fórmula ($tokens\ entrada \div 1M \times tarifa\ entrada$) + ($tokens\ salida \div 1M \times tarifa\ salida$). Esta web lo convierte en herramienta: eliges la tarea, el volumen y ves qué modelos encajan y cuánto costaría.

- **MVP:** tabla filtrable, calculadora, datos en un JSON con esquema validado en CI, cada dato con enlace a su fuente oficial.
- **Automatización:** un workflow programado (cron) abre cada mes una issue «revisar precios» y otro valida enlaces rotos.
- **Futuro:** tráfico por buscadores, newsletter, API de datos de pago.

Ojo con los derechos. No copies el PDF del curso: usa las tarifas de las páginas oficiales de cada proveedor y cítalas. Los precios de la guía son de una fecha concreta y algunos son promocionales.

Idea 3. Portfolio y CV desde tus repositorios de GitHub

Lee tus repositorios públicos con la API de GitHub y genera una página de portfolio y un CV en PDF compatible con sistemas de selección (ATS). Opcionalmente resume cada proyecto con IA en español.

- **MVP:** acción que ejecutas con un botón, genera `portfolio/index.html` y un PDF; se publica en Pages.
- **Diferencial:** español, CV imprimible y destacar solo los repositorios que cumplan un checklist de calidad.
- **IA opcional:** con la clave gratuita de Gemini guardada como `secret` de Actions. El plan gratuito tiene límites de uso y sus condiciones sobre los datos pueden diferir de las de pago; revísalas y no envíes código privado.

Mi recomendación de orden: 1 → 3 → 2. La primera te ahorra trabajo en todas las demás, la tercera te rellena el perfil y el CV, y la segunda es la de mayor potencial de tráfico pero la que más mantenimiento pide. Dime cuáles te convencen (o cámbialas) y me pongo con los repositorios.

Parte 3. Turnar OpenCode y Claude Code

Tienes dos herramientas con fortalezas distintas. Lo importante es que **no comparten memoria entre ellas**: lo que una aprende, la otra no lo sabe. Por eso todo el contexto va a archivos del repositorio (Parte 4) y Git es el punto de encuentro.

Reparto de papeles

	Claude Code	OpenCode
Acceso	Tu suscripción de Claude (permitido).	ChatGPT, Gemini con clave gratuita, modelos Free de Zen y, si procede, Copilot.
Úsalo para	Planificar, decisiones de arquitectura, fallos difíciles y revisar el trabajo.	Implementar tareas acotadas, documentación, tests y trabajo de volumen.
Coste	Consume la cuota de tu plan.	Modelos gratuitos o baratos: ahí gastas tokens sin miedo.

Cuentas de la empresa. Si ChatGPT o Copilot son licencias corporativas, confirma con IT que puedes usarlas en proyectos personales y desde OpenCode. La empresa puede ver ese uso.

Qué modelo para qué tarea (adaptado de la Guía de modelos 2026)

Tarea	Candidatos según la guía	Dónde usarlo
Aprender un concepto, documentar	Gemini 3.8 Flash, Sonnet 5, GPT-5.6 Terra	OpenCode (Gemini) o Claude Code
Implementar función o arreglar fallo acotado	Sonnet 5, Terra, GLM-5.3-Flash, DeepSeek V4 Flash	OpenCode o Claude Code
Trabajo multi-paso sobre el repo	GPT-5.6 Sol, Opus 5, Muse Spark 1.3, Kimi K2.7 Code	Claude Code (Opus) u OpenCode
Fallo difícil o arquitectura	GPT-6 Astra, Fable 5.1, Opus 5	Claude Code
Captura a interfaz	Gemini 3.8 Flash, Muse Spark 1.3, Qwen3.8-Flash	OpenCode
Cientos de transformaciones	GPT-5.6 Luna, Qwen3.8-Flash, GLM-5.3-Flash	OpenCode con modelo barato

Los nombres cambian; lo que dura son los criterios de la guía: filtra por requisitos, elige dos candidatos (uno económico y otro más capaz), Pruébalos con el mismo material y quédate con el que cumple a menor coste total. **Antes de subir de modelo, revisa el contexto y la especificación**, y no cambies modelo y esfuerzo de razonamiento a la vez, o no sabrás qué mejoró el resultado.

El ciclo de trabajo, paso a paso

1. **Plan (Claude Code).** Usa el prompt de arranque. Guarda el resultado en docs/PLAN.md con los hitos y sus pruebas, y anota las decisiones en MEMORY.md.
2. **Reparto.** Marca en el plan qué hitos van a cada herramienta. Los pequeños y bien definidos, a OpenCode.
3. **Ramas separadas.** Cada agente trabaja en su propia rama, y si van a la vez, en su propio directorio con `git worktree` (abajo). Así no se pisan los archivos.
4. **Ejecuta.** Al empezar, cada agente lee AGENTS.md y MEMORY.md. Pídele que trabaje un hito, escriba el test primero y pare al terminar.

5. **Verifica.** Los tests y el CI son el árbitro, no la opinión del modelo. Un «ya he ejecutado los tests» no cuenta hasta que lo ves en verde.
6. **Revisión cruzada.** Lo que hizo OpenCode lo revisa Claude Code (y al revés) sobre el diff: `git diff main...rama`. Pídele que busque errores reales, no que elogie.
7. **Integra.** Pull request, CI en verde, fusión con squash a main.
8. **Cierra la sesión.** Que el agente actualice MEMORY.md (estado, decisiones, siguiente paso) antes de salir. Esto es lo que permite que el siguiente agente retome sin contexto.

```
# Un directorio de trabajo por agente, sobre ramas distintas
git switch -c feat/hito-1
git worktree add ../mi-proyecto-opencode -b agent/opencode/hito-2
# ... trabajas en cada carpeta con su herramienta ...
git worktree remove ../mi-proyecto-opencode # al terminar y fusionar
```

Elige modelo con datos: el mini-experimento de la guía

La guía propone un ejercicio que merece la pena hacer una vez con tus dos herramientas: mismo proyecto, mismas instrucciones, dos modelos, y anotar en una tabla el resultado, los intentos, el tiempo, el coste y si *puedes explicarlo*. Repite si el resultado es dudoso: una sola respuesta acertada es una señal débil.

Parte 4. AGENTS.md y MEMORY.md en todos tus proyectos

Archivo	Qué es	Cuándo cambia
AGENTS.md	Las instrucciones estables del proyecto: comandos, estructura, convenciones, reglas. OpenCode lo lee automáticamente.	Pocas veces (cuando cambian las reglas).
CLAUDE.md	El archivo equivalente que lee Claude Code. Lo usamos solo para apuntar a AGENTS.md.	Casi nunca.
MEMORY.md	La memoria viva del proyecto: estado actual, decisiones y siguiente paso. Es una convención tuya, no un estándar.	Al final de cada sesión de trabajo.

No confundir. Claude Code también tiene su propia memoria automática (en tu carpeta de usuario). El MEMORY.md de esta guía es otra cosa: vive **dentro del repositorio**, se versiona con Git y lo leen todas las herramientas.

Paso 1. Crea AGENTS.md (una sola fuente de verdad)

Ambas herramientas tienen un comando que genera un borrador leyendo el proyecto: `/init`. Úsalo y luego recórtalo: lo que no cambie el comportamiento del agente sobra. Plantilla:

```
# AGENTS.md

## Qué es este proyecto
[Una frase. Para quién y qué problema resuelve.]

## Comandos
- Instalar:  npm ci
- Tests:    npm test
- Lint:      npm run lint
- Build:     npm run build

## Estructura
- src/      código
- tests/    pruebas
- docs/     plan y decisiones

## Convenciones
- Commits convencionales: feat:, fix:, docs:, test:, chore:
- Una rama por tarea; nunca commits directos a main.
- Cambios pequeños y con test.

## Reglas
- Nunca subir claves, .env ni datos personales.
- No añadir dependencias sin explicar por qué.
- Si algo es ambiguo, preguntar antes de suponer.

## Terminado significa
Tests en verde, lint sin errores y MEMORY.md actualizado.

## Contexto vivo
Lee MEMORY.md al empezar y actualízalo al terminar.
```

Paso 2. Crea MEMORY.md

```
# MEMORY.md
Última actualización: 2026-09-29 por [claude-code | opencode | yo]

## Estado actual
[Qué funciona hoy, en 3-5 líneas.]

## Decisiones (por qué)
- 2026-09-29: Usamos [X] en vez de [Y] porque [motivo].

## Siguiendo paso
1. [Tarea concreta, con el archivo y el test que la verifica.]

## Problemas conocidos
- [Descripción breve y cómo reproducirlo.]

## Registro de sesiones
- 2026-09-29 opencode: implementó [hito]; tests en verde.
```

- **Máximo unas 100 líneas.** Se carga en cada sesión y ocupa contexto (o sea, tokens). Lo antiguo, muévelo a docs/historial.md.
- **Cambios de rumbo entran en «Decisiones».** Evitan que el siguiente agente deshaga lo acordado.
- **Fechas absolutas,** nunca «ayer» o «la semana pasada».

Paso 3. Haz que ambas herramientas lo lean

Claude Code no lee AGENTS.md por sí solo, pero admite importar archivos. Crea CLAUDE.md en la raíz con solo esto:

```
@AGENTS.md
@MEMORY.md
```

Prefiere esto a un enlace simbólico: en Windows los enlaces simbólicos dan problemas.

OpenCode lee AGENTS.md automáticamente. Para que cargue también MEMORY.md, añade un opencode.json en la raíz del proyecto:

```
{
  "$schema": "https://opencode.ai/config.json",
  "instructions": ["MEMORY.md"]
}
```

OpenCode 2 es reciente y su configuración puede haber cambiado. Si instructions no surte efecto, consulta su documentación o pídele al propio agente que empiece leyendo MEMORY.md (escríbelo también en AGENTS.md: «Lee MEMORY.md al empezar»).

Paso 4. Comprueba que funciona

1. Abre el proyecto con cada herramienta y pregunta: «¿Qué comandos uso para los tests y cuál es el siguiente paso?».
2. Si contesta con lo que hay en AGENTS.md y MEMORY.md, está bien conectado. Si inventa, no los está leyendo.
3. Haz un cambio pequeño con una herramienta, pídele que actualice MEMORY.md, y comprueba que la otra lo ve.

Paso 5. Reglas personales para todos tus proyectos (una sola vez)

Lo que sea tuyo y no del proyecto (idioma de respuesta, sistema operativo, hábitos) va en un archivo global:

- Claude Code: `~/ .claude/CLAUDE.md`
- OpenCode: `~/ .config/opencode/AGENTS.md`

```
- Respóndeme en español.  
- Uso Windows 11 con Git Bash; da comandos compatibles.  
- No hagas push ni comandos destructivos sin preguntarme.  
- Explica brevemente el porqué de los cambios importantes.
```

Paso 6. Reutilízalo: plantilla de repositorio

En GitHub, crea un repositorio con estos archivos y márcalo como **Template repository** (Settings). Cada proyecto nuevo se crea con «Use this template». Cuando tengas la Idea 1, esto se automatiza con un comando.

Errores frecuentes

- AGENTS.md enorme y lleno de obviedades: el agente lo ignora y gastas contexto.
- MEMORY.md que nadie actualiza: peor que no tenerlo, porque miente. Que actualizarlo sea la última instrucción de cada sesión.
- Duplicar las mismas reglas en CLAUDE.md y AGENTS.md: se desincronizan. Un único origen e importación.
- Meter secretos: estos archivos van al repositorio.

Parte 5. Control de versiones y CI/CD desde el primer día

Git: reglas mínimas

- **main siempre funciona.** Se trabaja en ramas: feat/..., fix/..., agent/opencode/....
- **Commits convencionales** (feat: añade init). Dan un historial legible y permiten generar el changelog solo.
- **Todo entra por pull request**, aunque trabajes solo: deja rastro y ejecuta el CI.
- **Etiquetas de versión** (v0.1.0) y GitHub Releases con notas.

Protege la rama main (gratis en repos públicos)

1. En el repositorio: Settings → Rules (Rulesets) → New branch ruleset, sobre la rama main.
2. Activa: exigir pull request y exigir que pasen las comprobaciones (tu workflow «CI»).
3. Activa además Dependabot y el escaneo de secretos (Settings → Advanced Security / Code security).

Un workflow de CI base para Node

Archivo `.github/workflows/ci.yml`. Prueba en Windows y Linux y en dos versiones de Node. Comprueba en la documentación de cada acción cuál es la versión mayor actual.

```
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
        node: [20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm test
      - run: npm run build
```

Despliegue (CD) a GitHub Pages

Cuando el proyecto tenga una web o documentación, añade un segundo workflow que se ejecute solo al actualizar main, construya el sitio y lo publique en Pages (Settings → Pages → Source: GitHub Actions). GitHub ofrece plantillas ya hechas en la pestaña Actions del repositorio.

Publicar un paquete (si es una CLI)

1. Crea una cuenta en npm y genera un token de publicación.

2. Guárdalo como *secret* del repositorio, nunca en el código.
3. Crea un workflow que se dispare al publicar una Release y ejecute `npm publish`.

Los tokens y claves los creas y guardas **tú**. Yo puedo dejar los workflows preparados, pero no debo introducir ni ver tus credenciales.

Parte 6. Checklist final

Antes de dar un repositorio por terminado

- README con qué es, captura/GIF, instalación, uso y enlace a la demo
- LICENSE, descripción y topics
- CI en verde y con badge; main protegida
- AGENTS.md, CLAUDE.md y MEMORY.md actualizados
- Al menos una Release etiquetada con notas
- Sin secretos en el historial
- Puedes explicar cada parte del código sin ayuda
- Añadido al README de tu perfil y a tu CV

Siguiente paso

Elige tus tres ideas (las propuestas u otras) y dime en qué orden. Con eso, el trabajo se reparte así:

- **Yo (Claude Code):** plan de cada proyecto, estructura del repositorio, AGENTS.md/CLAUDE.md/MEMORY.md, CI/CD, plantillas de issues y PR, y la revisión del trabajo.
- **OpenCode:** implementación de hitos acotados y documentación, en su propia rama, revisada después.
- **Tú:** las decisiones de producto, las credenciales (claves, tokens de npm, secretos de Actions) y la aprobación de cada pull request.